





老猿说系列

说说sharding那些事儿(下) --订单交易平台实践

我是谁

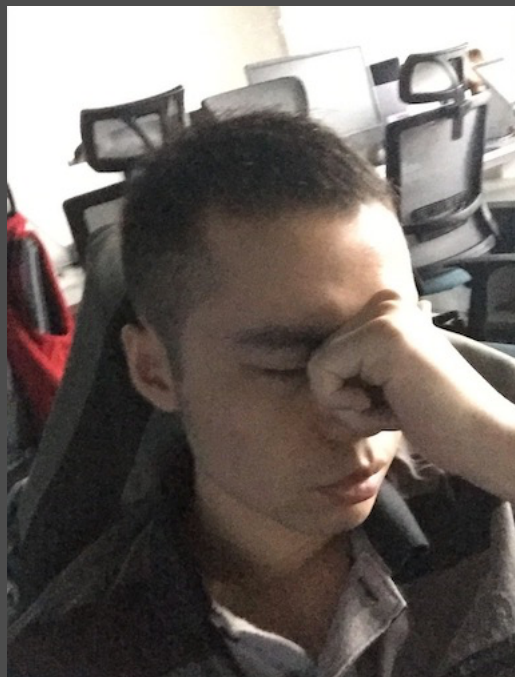
- ◎ 冯文
- ◎ 84年生人
- ◎ SNS 培训 通信 游戏 音视频
- ◎ 15-4加入客如云同步组，现负责订单交易平台&商品中心

我是谁

进公司前的我



进公司后的我



前情回顾

- ◎ 背景&技术选型
- ◎ 问题&方案：全局ID、跨库查询、数据迁移
- ◎ 分布式数据访问组件基本设计思路及在客如云sharding方案中地体现

演进历程

2017-3



调研&选型

2017-7



订单库垂直分库

2017-12



分区部署&UNID

2018-3



稳定区上线

2018-6



新增第二片

2018-9



列表分区&全链路压测

2019-6



新增第三片

现状

截止19-8

日均465W+

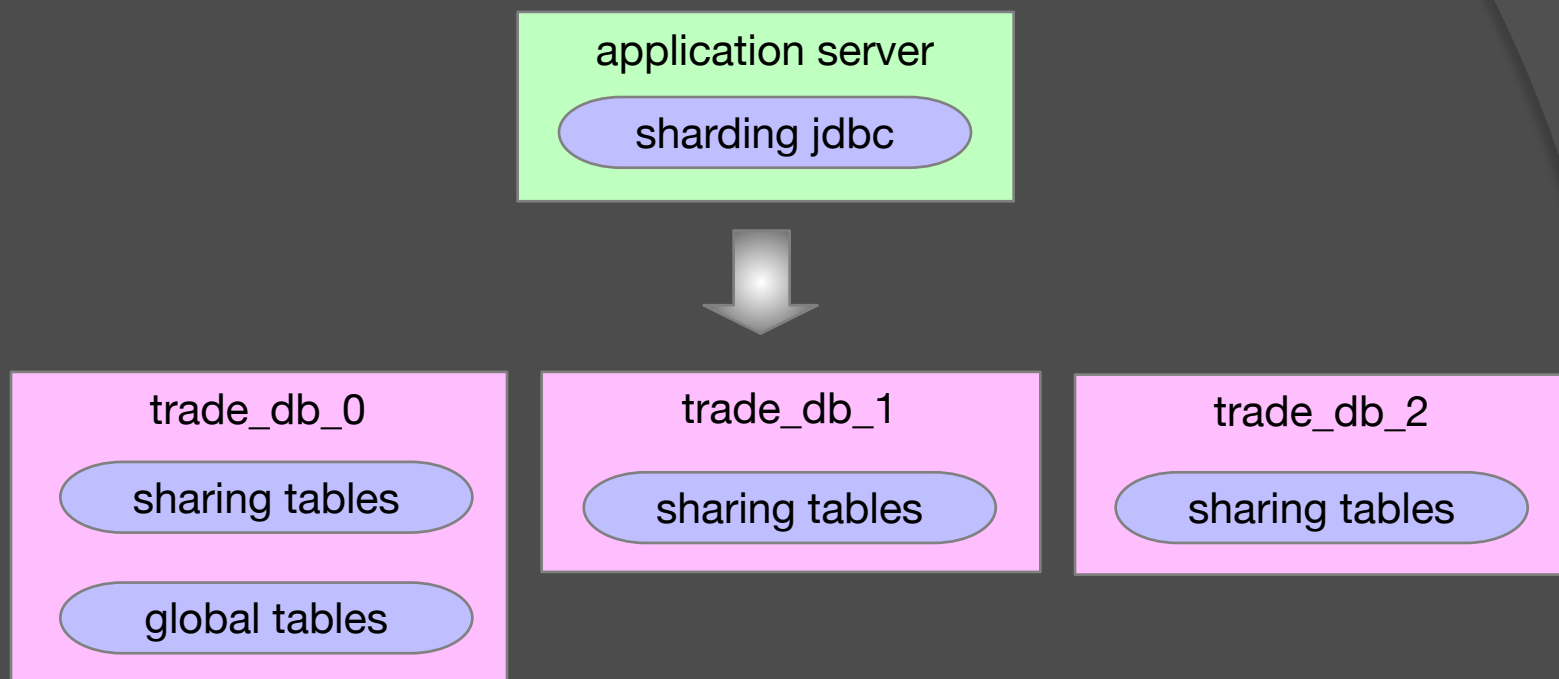
3片

5W+品牌/片

平均存量2KW+

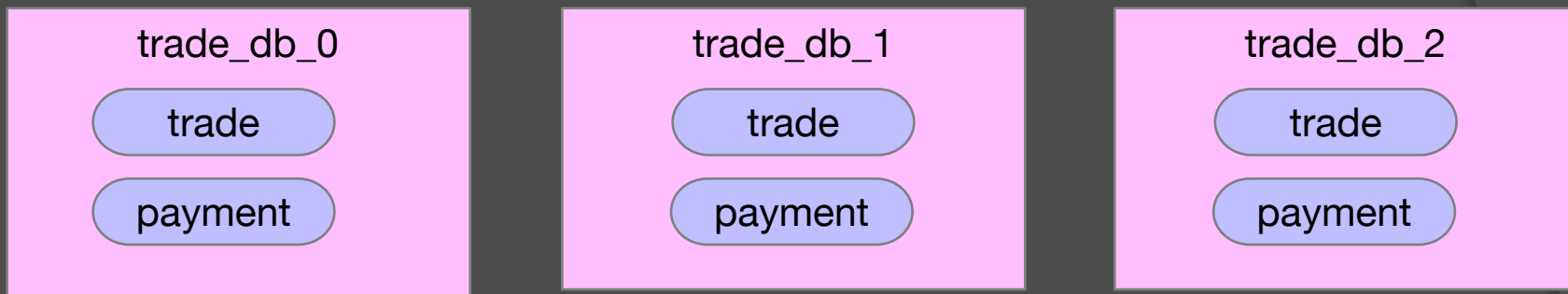
CPS1W

现状



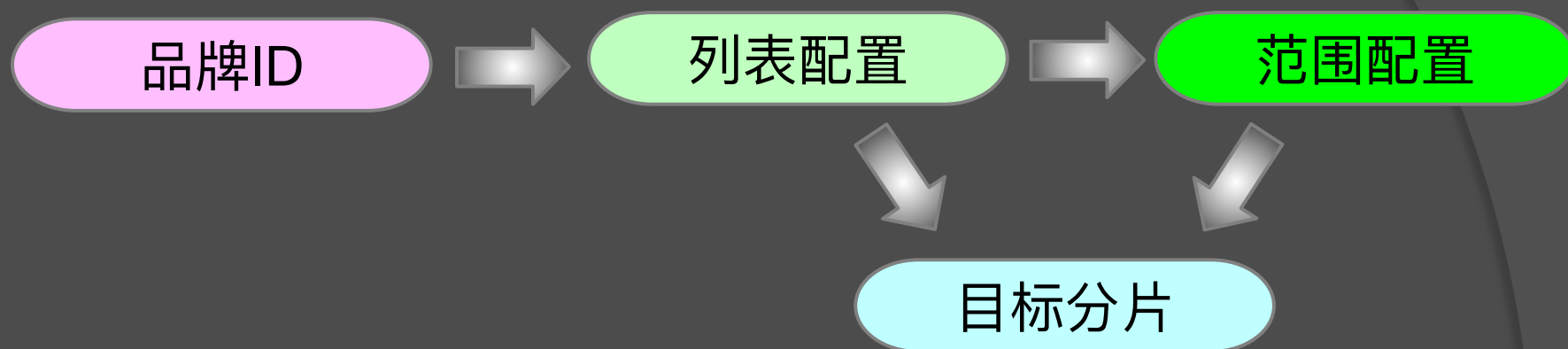
- global tables:不需要sharding的表，物理上可以设计为独立schema

全局策略



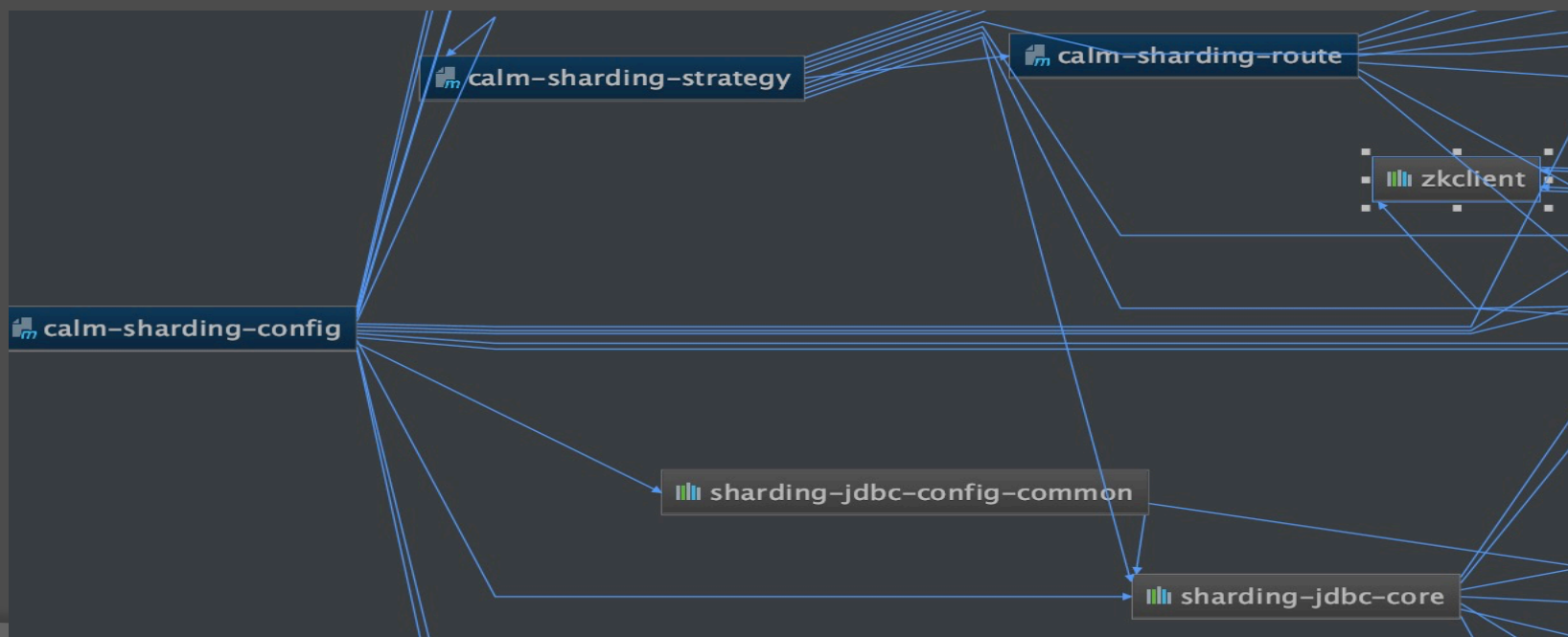
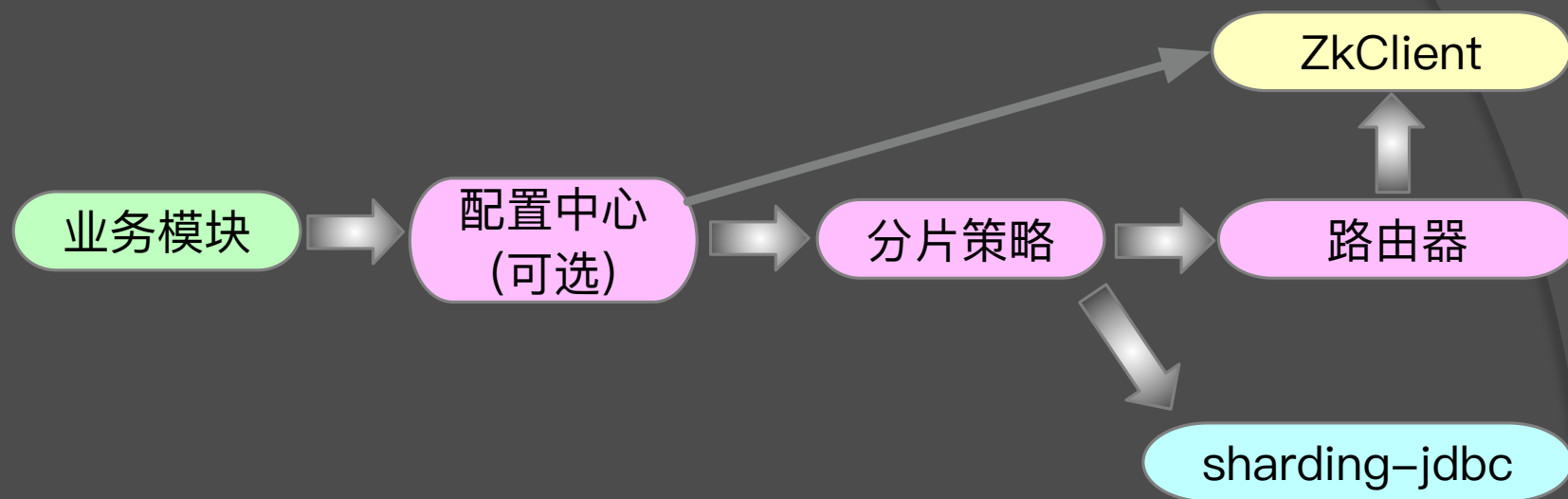
- 只分库，不分表
- 按照品牌ID进行范围分片
- 所有片中的表名均相同
- 优点：平滑扩容，避免数据迁移
- 缺点：每个片中的数据量不均匀

分片策略



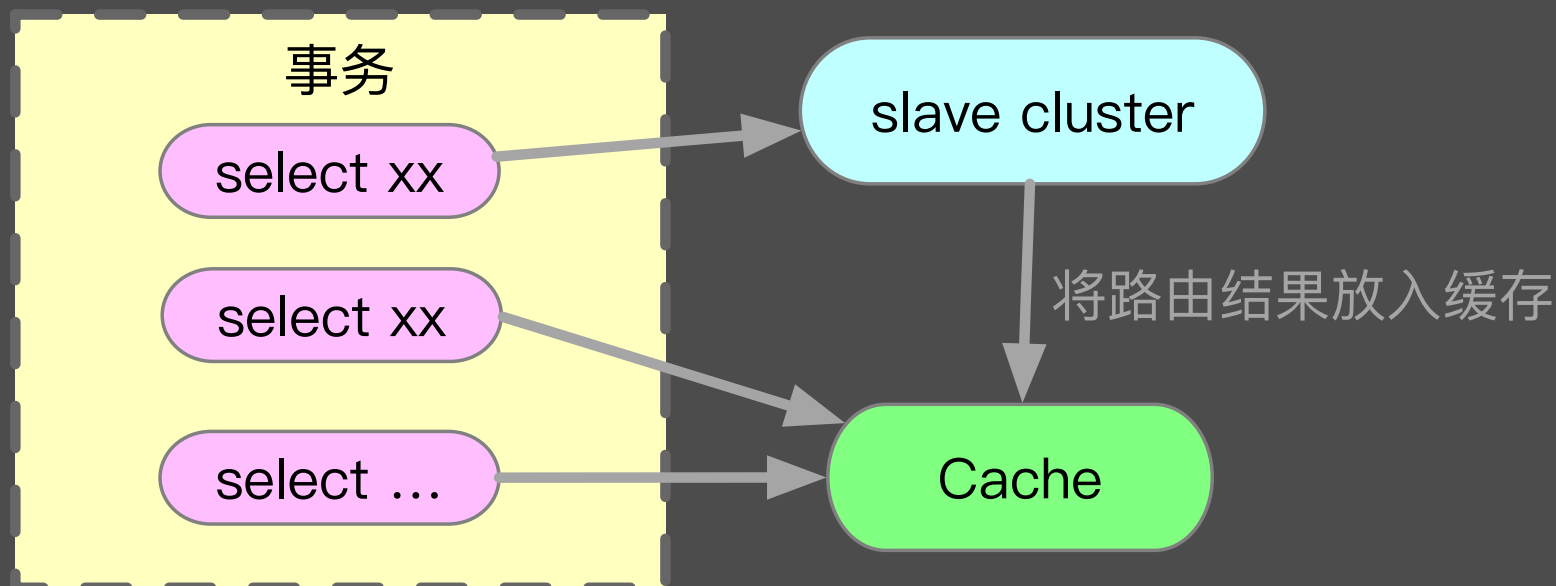
- 范围分区+列表分区（热点配置）
- 优先热点商户配置
- 可扩展的自定义分片策略，预留二级sharding key（如品牌ID+门店ID）
- 缺点：从存量数据中拆分热点商户需要做数据迁移
- 目前列表分区仅用于全链路压测

模块设计



事务

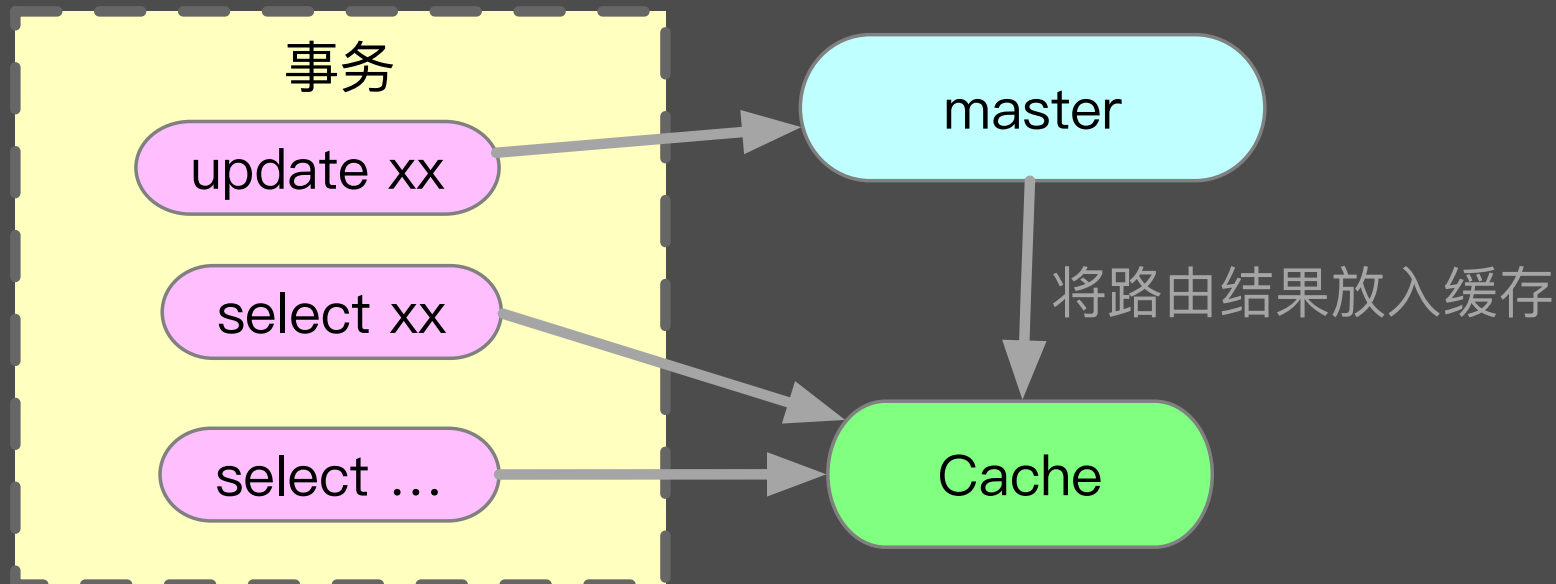
◎ 一个事务中多次读



- ◎ 均使用第一次的从库路由结果
- ◎ 隐患：一个线程中若开启多个读事务会分别路由到不同的从库

事务

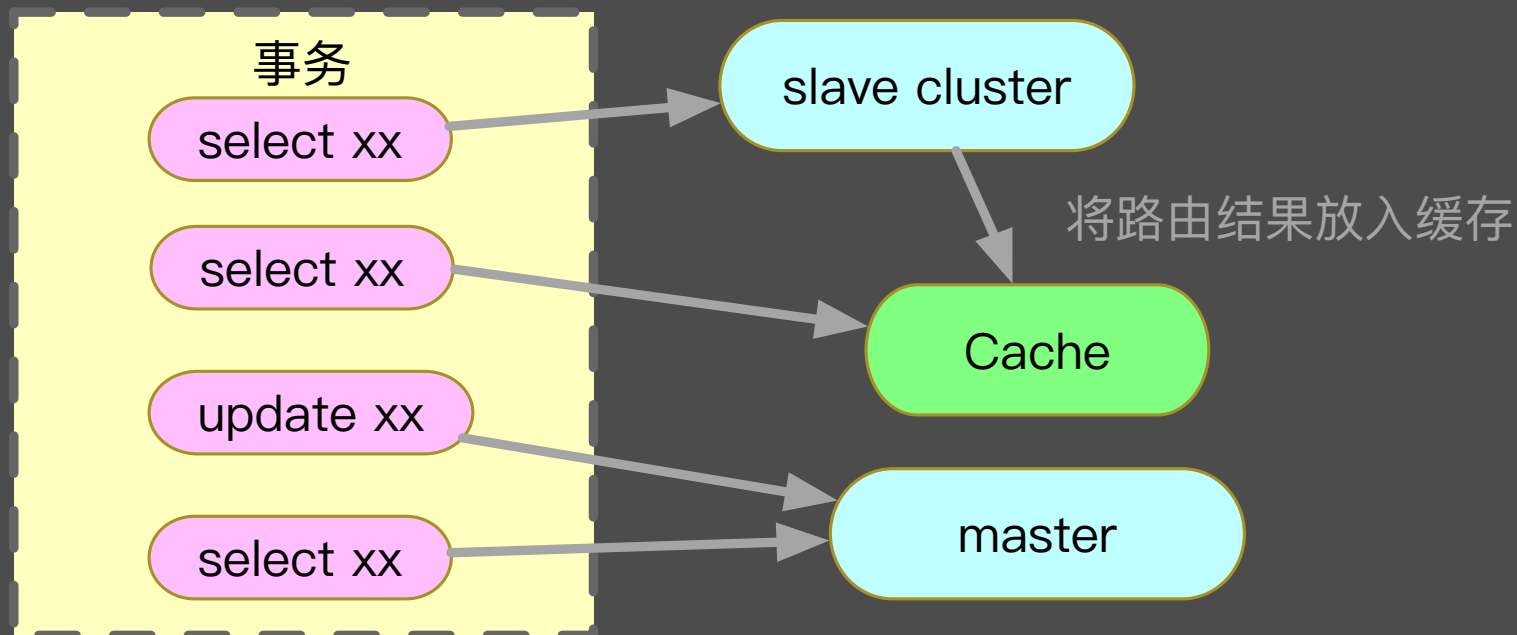
◎ 一个事务中先写后读



◎ 基于hint判断为DML，后续操作全部走主库路由

事务

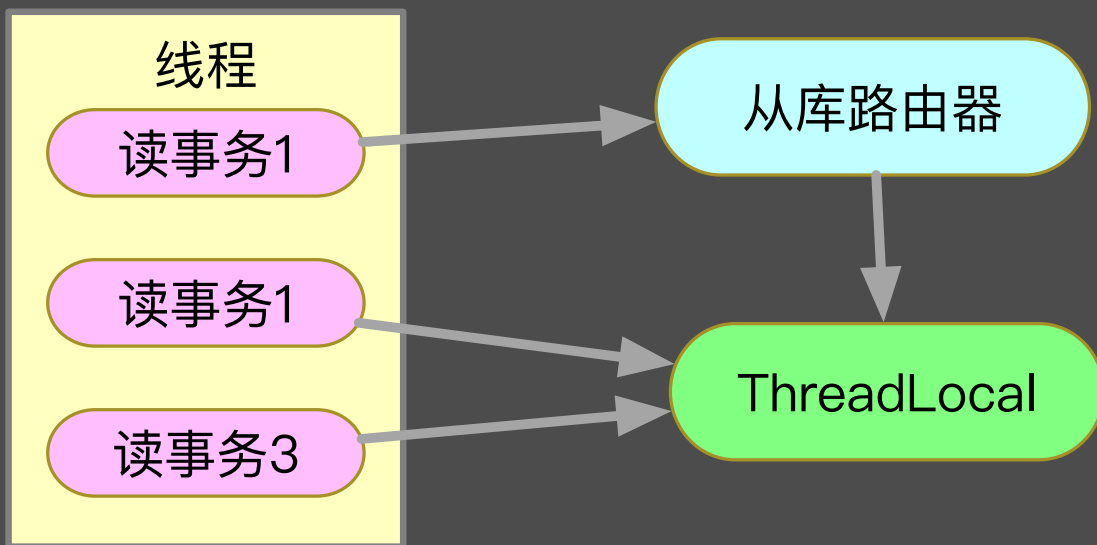
◎ 一个事务中先读再写



- ◎ 隐患：不适合个别强一致性场景，如所有查询均需要查到最新的数据

定制化-多从库路由

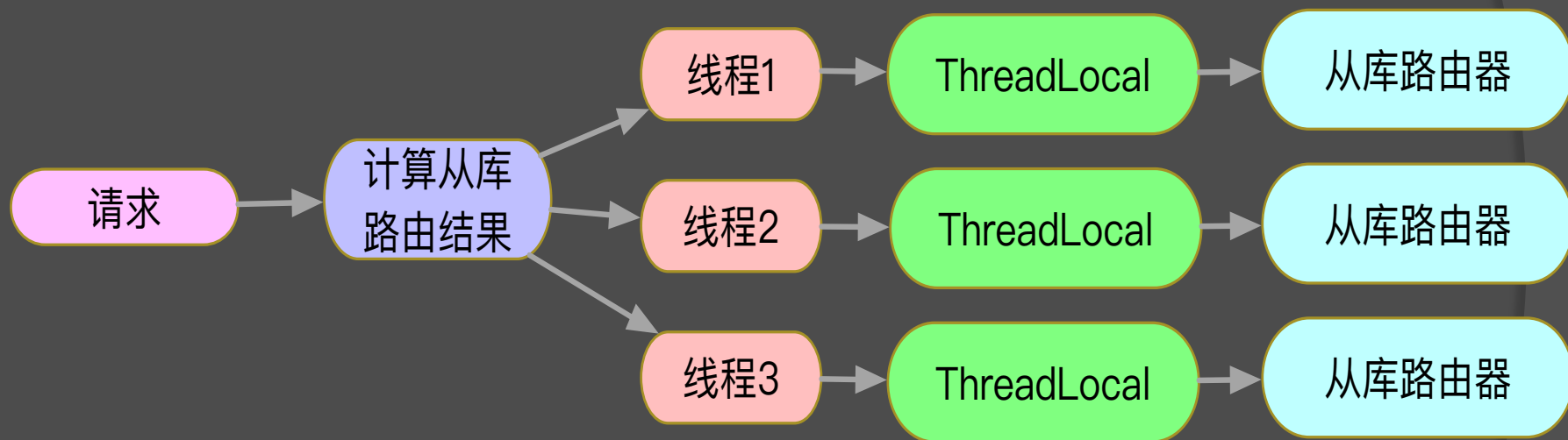
- 保证一个线程中所有读事务路由到同一个从库



- 修改源码，支持自定义多从库路由策略

定制化-多从库路由

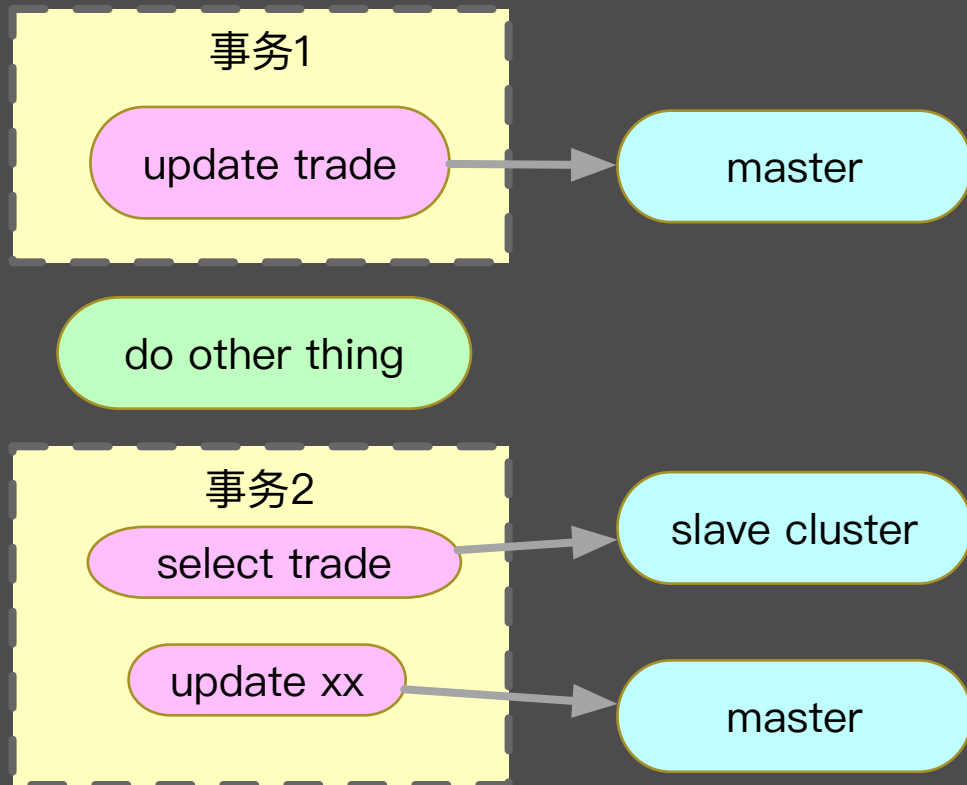
- 保证一个请求中所有线程的读事务路由到同一个从库



- 提前计算从库路由结果

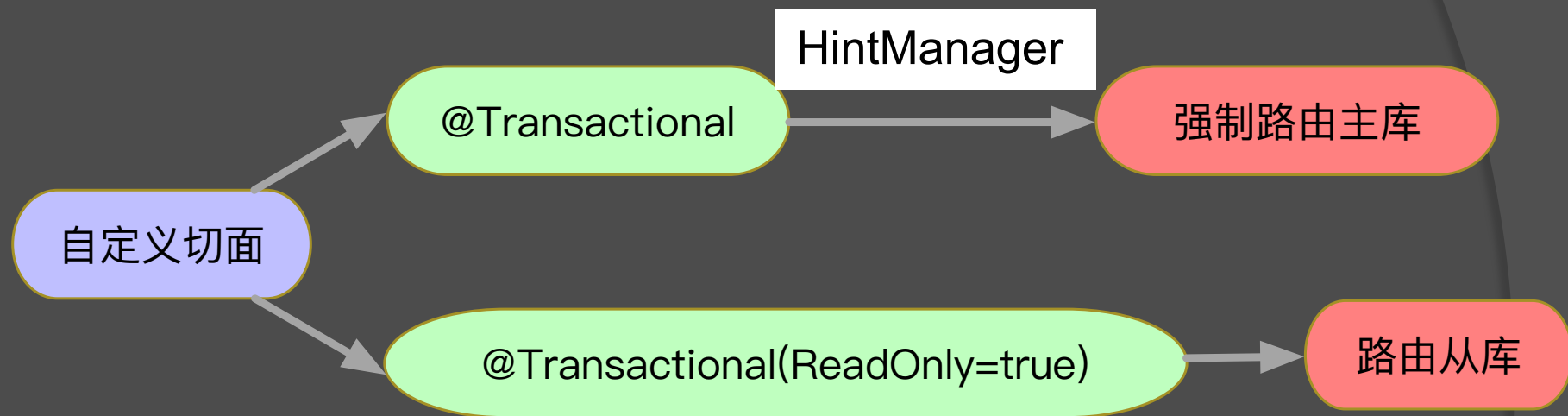
定制化-读写分离

默认基于hint路由的隐患



不支持强一致性场景:事务2必须全部路由主库

定制化-读写分离



- 人为通过声明事务控制

定制化-配置中心

◎ 原有基于xml配置问题

- inner bean重复初始化，数据库连接数额外占用
- 无法实现配置外置化

◎ 方案

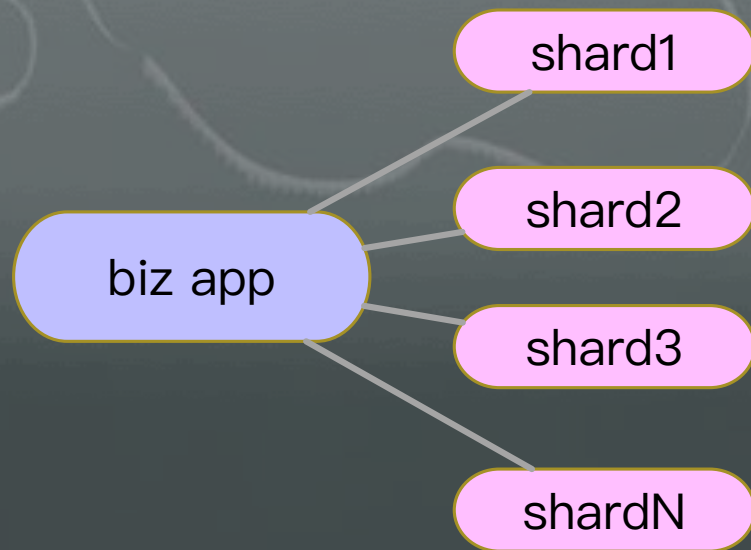
- 升级版本至3.x，使用sj-spring-boot-config
 - 1.x~3.x改动很大，回归验证
 - table-rule的配置策略过于复杂，会导致我们的ZK配置膨胀
 - 强依赖sb
- 自研配置组件

```
sharding.ds1.xx=  
sharding.ds2.slave3.xx=  
sharding.database.strategy1=
```

.....

一些思考

- 随着业务系统节点数量增长，数据库连接数终将达到瓶颈



抛砖引玉

- 应用架构继续细化，仅保持尽可能少的服务需要访问shard
- 增加自定义前置路由，每个服务只访问固定的某个shard

Thank
谢谢观赏

fengw@keruyun.com

了解更多

- <https://conf.shishike.com/x/7C-u>
- <https://conf.shishike.com/x/NTru>

